

Engineering Assignment

Deadline: 3 days from the day you receive this assignment.

What to build

A web app for analysing legal contracts.

A user uploads a contract (PDF or DOCX) and asks questions about it in a chat. The app answers using only what is in the document, and backs every answer with the exact quote it came from. Clicking a quote takes the user to that passage in the document.

No login or account system is needed. Assume a single user.

Part A: Core features

1. Document upload and processing

- Accept PDF and DOCX. Reject other file types with a clear message.
- Extract the text and store it.
- Show processing status while it happens. The user should never be left wondering whether anything is happening.
- Handle a scanned PDF with no readable text properly: tell the user, rather than saving an empty document and treating it as successful.
- A document library where uploaded files are listed, opened and deleted.

2. Chat with a document

- Ask questions about an uploaded document and get answers.
- Answers **stream in as they are generated**, not all at once at the end.
- The user can stop an answer while it is being written, and whatever was generated is kept.

- Chat history is saved per document and can be reopened.

3. Verified quotes

This is the most important requirement in the assignment.

Every answer must be supported by quotes from the document. **Before a quote is shown to the user, your code must confirm that the quote actually exists in the document text.**

- Quote found: show it as verified, and let the user open it in the document.
- Quote not found: the AI invented or paraphrased it. It must never be presented as if it were genuine. Remove it, or mark it clearly as unverified.
- Do not trust any position, page number or offset the AI reports. Locate the quote in the text yourself.
- Allow for whitespace differences. Text extraction adds and removes spaces and line breaks without changing the words, so an exact string match will wrongly reject quotes that are genuinely present.

If the answer is not in the document, the app must say so instead of inventing one.

4. Large documents

A 150-page contract must work. It will not fit into a single AI request, so you will need a strategy for splitting it. The approach is your choice.

One rule: **if your app only read part of a document, it must not answer as though it read all of it.** Confidently stating that a clause does not exist, after reading only the first 30 pages, is the worst possible output.

Part B: Advanced features

Required. These are harder than Part A, and we will be looking at whether they actually work, not just whether they are present.

5. Citation highlighting

Clicking a quote in an answer opens the document, scrolls to that passage and highlights it.

The extracted text and the rendered page are laid out differently, so the mapping takes real work. Handle quotes that span multiple lines, cross a page break, or appear more than once.

6. Multi-document questions

The user selects several documents and asks one question across all of them.

- Each quote identifies which document it came from.
- The answer compares across documents rather than listing separate answers.
- Each quote is verified against its own document, not the whole set.

7. Document comparison

Upload two versions of a contract and see what changed.

- Differences at clause or paragraph level, not a character diff.
- A plain-language summary of what changed in substance. A reworded sentence is not the same as a liability cap moving from AED 100,000 to AED 1,000,000.
- Filter or sort by how significant a change is.

Part C: Choose one

Pick **one** of the two challenges below. Both are deliberately harder than anything above, and we do not expect everyone to finish. An honest partial attempt with a clear account of your approach and what defeated you is worth more than skipping this section. Tell us in your note which you picked and why.

Option 1: Tracked-change redlining

The user asks for a change in plain language ("make the liability cap mutual"). The app writes the revised wording back into the .docx as **real tracked**

changes, so it opens in Word with insertions and deletions the user can accept or reject individually.

- The downloaded file opens cleanly in Word or LibreOffice, with the edits shown as revisions the user can accept or reject, not as plain text that has already been changed.
- All original formatting survives: fonts, bold, numbering, tables, styles.
- Only the changed text is touched; nothing else is reformatted or renumbered.
- Multiple edits can be applied in a single pass.

The text you need to change is usually split across several internal fragments with different formatting, so a plain string replacement will not work.

Regenerating the whole document and diffing it appears to work and then produces hundreds of spurious changes. That does not count.

Option 2: Agentic document research

Instead of pushing document text into the prompt, give the model tools it can call, such as `search_document(query)` , `get_section(number)` and `list_clauses()` , and let it decide what to look up before answering.

- A real multi-round loop: the model can call tools, read results, and call more.
- Show what it is doing as it happens ("Searching for termination provisions..."), not a spinner.
- A hard cap on rounds, so it cannot loop forever or run up an unbounded bill.
- Malformed or invented tool calls (wrong name, missing arguments, nonsense parameters) handled without crashing the request.
- Verified quotes still apply to the final answer.

Tech stack

Next.js preferred. Beyond that, use whatever stack you are comfortable and fast in. The result matters more than the tools.

For the AI, use any provider (OpenAI, OpenRouter, Anthropic, Gemini, or a local model). Read the API key, base URL and model name from environment variables. Use an inexpensive model; model quality is not being assessed.

Do not commit API keys to the repository.

Extras (optional, bonus points)

Only if Parts A, B and C are complete and working. An unfinished extra does not help you.

- **Anonymise.** Replace names, companies, emails and phone numbers with placeholders like [PERSON_1] , consistently throughout, with a mapping that can reverse it.
- **Semantic search.** Embeddings so questions retrieve the most relevant passages instead of scanning the document linearly.
- **Export.** Download an answer with its verified quotes as a formatted PDF or Word document.
- **Clause extraction.** Automatically identify and list standard contract clauses (termination, liability, governing law, confidentiality).
- **Arabic support.** Arabic text extraction and right-to-left layout.
- **Background processing.** Recovers if the server restarts mid-job.
- **Voice input.** Ask questions by speaking.

How we evaluate

Our primary focus is the application you actually built. We will open your deployed link and use it ourselves, uploading our own contracts, asking our own questions, and testing each feature properly. How the app behaves in our hands is the main basis for our assessment, so make sure the deployed version is the real, working thing and not a partial build.

Alongside that, we review the **GitHub repository** and the **README**, which are both taken into consideration.

What we value throughout is **accurate functionality** and **quality of work**.

We also assess:

- **Interface quality.** The app should look and feel finished. Considered layout and typography, clear states for loading, empty and error conditions, and an interface someone could use for real work without being walked through it. This is part of the assessment, not an optional polish step.
- **Judgement.** What you chose to build, what you chose to leave out, which Part C option you picked, and whether you can explain all of it.
- **Documentation.** The README and demo video.

Counts against you: claiming something works when it does not, a committed API key, or an app that only holds up on a small sample file.

What to submit

1. **GitHub repository link.**
2. **Deployed link.** A working, live version we can open and use. Vercel, Railway, Render or [Fly.io](https://fly.io) are all fine.
3. **README** containing:
 - What the app does
 - **Screenshots** of the main screens: upload, chat with verified quotes, citation highlighting, document comparison
 - How to run it locally
 - What is finished and what is not
4. **Demo video (3 to 5 minutes).** A screen recording of you using the app: upload a document, ask a question, show the quote verification working, click through to a highlighted citation, demonstrate the comparison feature, and show your Part C work, including what does not yet work, if anything. Loom, Google Drive or an unlisted YouTube link.
5. **A short note** (half a page) covering:
 - How your quote verification works, and where it could fail
 - How you handled large documents
 - Which Part C option you chose and why, how far you got, and what the hardest part of it was
 - What you would build next with more time